ATLAS

How we build it: the stack, the AR/VR pipeline and everything we need to implement

Blades	Hub + Bend Your Room · Breath Room · Lung Mission · Post Circling · FieldFit · House Call
Interaction	technique registry · twin inputs · Geometry Lens · Two Compasses · One-Euro filters
Non-Euclidean engine	IGeometry (E · P · T · H · S · B · 4D · warp) · charts + gluings · tile solver · H^2 layout
Rendering	one camera, two eyes · stencil portals · Klein shader · 4D slicer · sky compute · final pass + late warp
Sensing + safety	ARKit 6DoF · LiDAR depth · Vision hands (ANE) · cube · Core Motion prediction · safety FSM
Platform	Unity 6.3 LTS · URP 17 · AR Foundation 6 (ARKit) · IL2CPP · Metal · Swift plugins · Cardboard optics

WHAT THIS IS

The engineering plan in plain language: which technologies and libraries we use, how a frame gets from the camera to your eyes, how the code is organised, how data is prepared, and how we build, test and ship it.

COMPANION DOCUMENTS

Design: `ATLAS_Plan.pdf` and `ATLAS_Implementation_Brief.md`.
Engineering: this PDF and `ATLAS_Engineering_Spec.md` (full detail for the implementation agent).

HARD TARGETS

60 fps stereo · safety on the phone · ≤8 ms GPU · ≤30 ms motion-to-photon · ≤1.8 GB memory

5 deliverables: phone app, partner client, server, spectator, pipelines

1 camera renders both eyes

16.7 ms per frame at 60 fps

10 assemblies with enforced boundaries

24 Phase 0 spikes on the real hardware

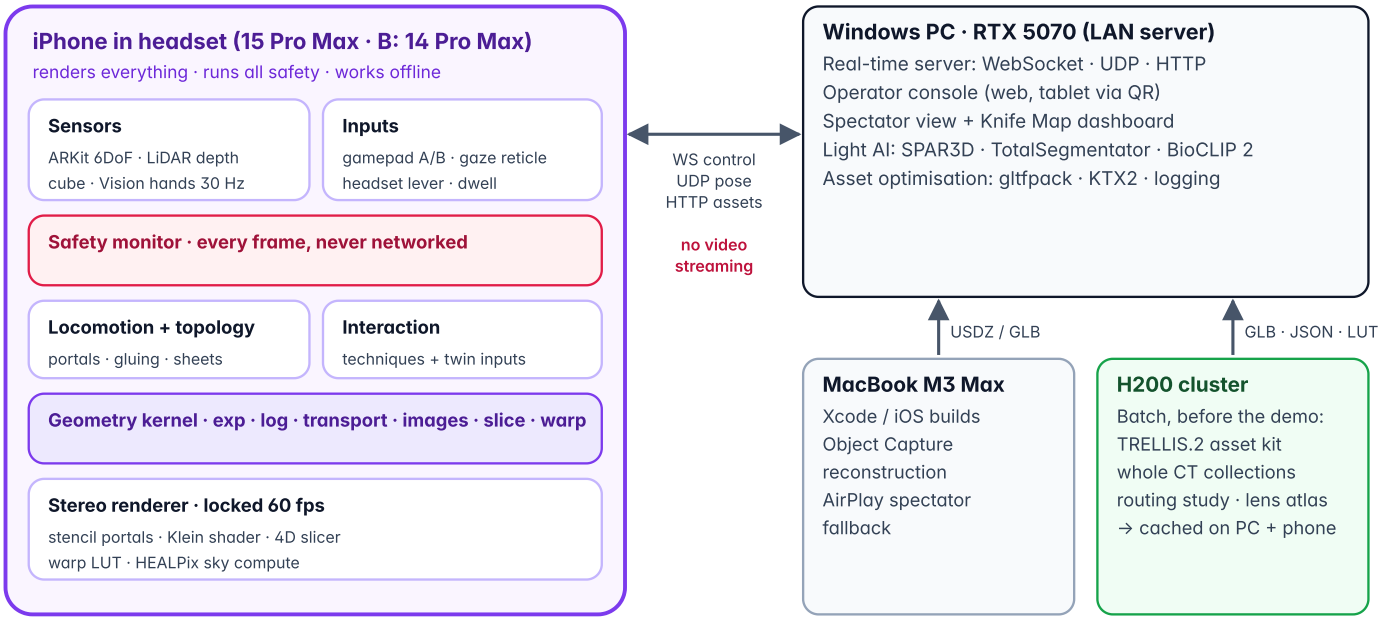
≤20 min from commit to both phones

01 · WHAT WE BUILD

Five deliverables, one phone does the experience

The phone renders everything and runs all safety. The PC prepares, logs and shows; the H200 cluster prepares data in advance. Nothing the user feels waits on the network.

Deliverable	Runs on	Built with	Job
Phone app	iPhone 15 Pro Max (A), 14 Pro Max (B)	Unity 6.3 LTS, URP, AR Foundation (ARKit), Swift plugins	Tracking, safety, stereo rendering, interaction, blades, logging
Partner client	Galaxy S24	Same Unity project, ARCore, minimal scene	House Call option: partner pose, avatar, voice
PC server + web console	Windows, RTX 5070	Python 3.11, FastAPI, msgpack; TypeScript + D3	Link, assets, Roll a Lung builds, object meshes, console, dashboard, logs
Spectator renderer	Windows	Same Unity project, spectator build	Re-renders the judge's view from replicated state
Pipelines + batch	PC and H200	Python package `atlas-pipelines`, Apptainer, Slurm	Case cards, greedy study, asset kit



Rules that shape the code

Safety never touches the network. The safety module cannot even reference the networking module (enforced by the build). It runs first in every frame and never waits.

The slower phone sets the bar. Every budget and test runs on the iPhone 14 Pro Max; both phones use its quality tier, so every judge gets the same experience.

Every hand gesture has a twin. Gamepad, gaze dwell, the cube or the console can do anything hands can, so the demo works in bad light.

No frames stored. Camera images are processed in memory only; logs are pseudonymous; no clinical claims in any string.

02 · THE STACK

What we use, layer by layer

Layer	We use	Why	Not chosen
Phones	iPhone 15 Pro Max (A), 14 Pro Max (B); Galaxy S24 partner	LiDAR depth, RoomPlan, Vision, strong sustained GPU	Android as the headset phone (no depth sensor)
Headset	Camera-window phone headset; Cardboard viewer profile	Camera and LiDAR stay clear; exact lens distortion	Zapbox controllers (can't run alongside ARKit)
Engine	Unity 6.3 LTS, URP 17, IL2CPP, Metal	Tooling, AR Foundation, Android from the same project	Unreal (no Cardboard path), Godot (thin XR tooling), WebXR (off on iOS)
AR tracking	AR Foundation 6 + ARKit (ARCore for the partner)	6DoF, depth, meshing, image tracking, camera frames	Cardboard tracking (rotation only)
Stereo	One camera, side-by-side render pass; Cardboard C API for optics	Unity has no single-pass stereo on iOS	Two eye cameras (CPU cost); Cardboard XR plugin (1-day spike)
Latency	Core Motion 200 Hz prediction + late rotational warp	ARKit gives the pose at camera capture time	Late latching (not on this path)
Native iOS	Swift plugins: Vision (Neural Engine), RoomPlan, Core Motion, world map, thermal; Apple Unity plugins: haptics, gamepad	First-party APIs, best performance	MediaPipe on iOS (build friction)
Maths	Burst + Unity.Mathematics; double precision on the CPU	Curved space needs precision; jobs are fast	Float world coordinates
Assets	glTF + meshopt + KTX2 via glTFast; gltfpack, toktx	Small, fast to stream and decode	Gaussian splats on the phone
Async + wire	UniTask; MessagePack v3 (generated, IL2CPP-safe)	No garbage per frame; compact	JSON (size); protobuf (later if needed)
Server	Python 3.11, FastAPI, asyncio UDP, msgpack, SQLite	Quick to build; one process	—
Console	Vite + TypeScript + D3 web app	Any tablet via a QR code	Unity UI
Spectator	Windows build of the same Unity project	Same renderer; fed by state, not video	Streaming video from the phone
Multi-user	LiveKit voice; UniVRM avatars	Open source; iOS and Android	ARKit collaboration (iPhone only); cloud anchors (internet)
AI	SPAR3D on the 5070 (live object); TRELLIS.2 on the H200 (kit); rembg / SAM 3	Fits 12 GB live; best quality offline	TRELLIS.2 live on the 5070 (needs ≥24 GB)
Medical	TotalSegmentator; MONAI / nnU-Net airways; skan; SimpleITK; rt-utils; astropy-healpix	Open licences; proven tools	GPL tools (kimimaro, healpy) in shipped code
Signing	Free Apple account in Xcode ("Personal Team")	No enrolment, fee or review wait; covers both iPhones	Paid Developer Program (fallback if spike 0.26 fails)
Build + test	GitHub Actions on the M3 Max, xcodebuild + devicectl, Unity Test + Graphics Test Framework, pytest, Sentry	Commit to both phones in ≤20 min	Cloud macOS runners (cost); TestFlight (needs the paid program)

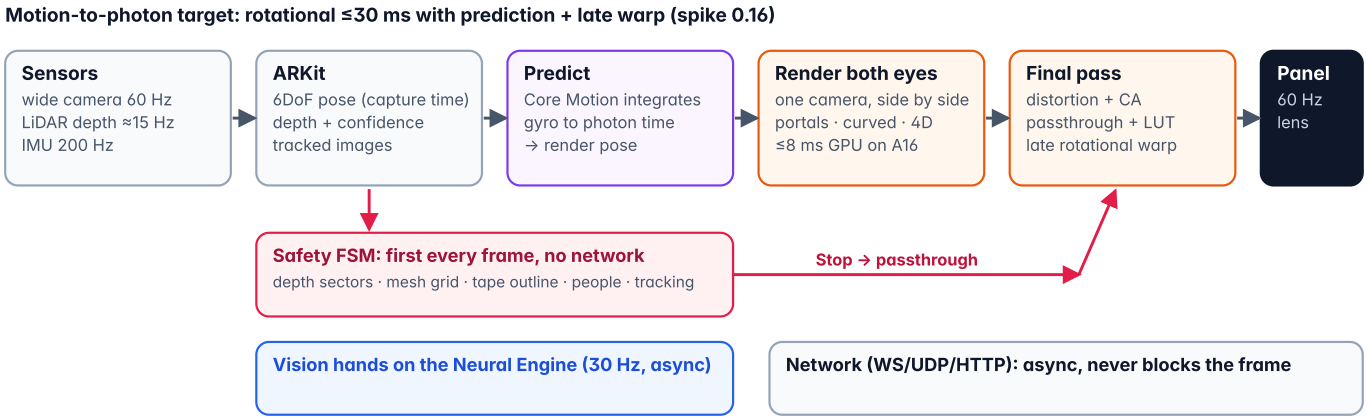
How the pieces talk. C# calls Swift through C entry points (`@_cdecl` / `DllImport("__Internal")`). Phone and PC use WebSocket (control), UDP (pose) and HTTP (assets), all MessagePack from one schema. The H200 sends content-hashed case packages to the PC by rsync.

Licences. MIT, Apache or BSD by default (UniTask, MessagePack, glTFast, meshoptimizer, KTX, LiveKit, UniVRM, TotalSegmentator, MONAI, skan, astropy-healpix, TRELLIS.2). Check first: SAM 3 (gated), SPAR3D (community). Not in shipped code: kimimaro, healpy (GPL).

03 · THE AR/VR PIPELINE

From the camera to your eyes

A frame starts as a camera image and IMU readings and ends as two lens-corrected images on the phone's screen, 16.7 ms later. Safety runs first; hand tracking and networking run alongside and never hold the frame up.



AR session settings

Camera	60 fps format (e.g. 1920×1440); no camera background in VR scenes
Depth	Environment depth, smoothed, with confidence (≈15 Hz)
Meshing	On while mapping the room, then off (saves heat)
Planes	Floor only, during calibration
Images	6 cube faces + floor marker; ≤2–3 moving
World map	Saved after the venue scan; reloaded at start

Frame loop (order matters)

1. Poll inputs: pose, depth, cube, gamepad, latest hands
2. **Safety**: depth sectors, mesh grid, tape outline, people, tracking
3. Locomotion: portal, wall and cut crossings; chart changes
4. Interaction techniques call the geometry kernel
5. Blade simulation
6. Render both eyes; final pass with the newest pose
7. Log to a ring buffer (never blocks)

> **Prediction:** the gyro is integrated from the camera frame's timestamp to the expected photon time, and applied twice: once when rendering, again as a late warp.

Lens optics: Cardboard C API (v1.35.0)

Pair once	`CardboardQrCode_scanQrCodeAndSaveDeviceParams()` scans the headset's QR code
At start	`CardboardQrCode_getSavedDeviceParams` → `CardboardLensDistortion_create` for the panel size
Per eye	`…_getEyeFromHeadMatrix`, `…_getProjectionMatrix`, `…_getDistortionMesh` → a Unity mesh, uploaded once
Not used	Cardboard's head tracker (ARKit gives 6DoF) and its distortion renderer (our final pass adds the warp, passthrough and LUTs)

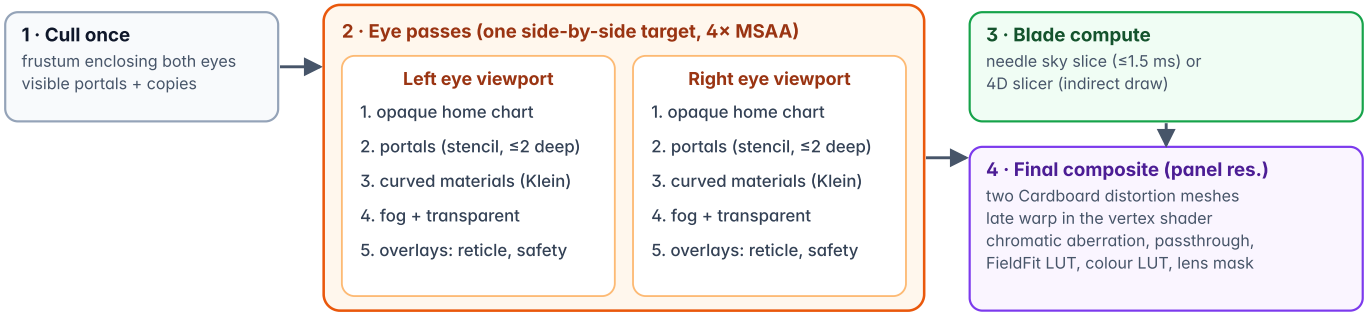
Passthrough and Safe Harbour

1. ARKit's camera image arrives as Y and CbCr Metal textures; the final pass converts them to RGB.
2. It is undistorted with that frame's camera intrinsics and placed on a plane 1.5–2 m ahead (optionally pushed out per pixel from smoothed depth).
3. It is rotated by the same late-warp correction as the scene, so it doesn't swim.
4. **Safe Harbour** = passthrough + room outline + play-area ring, all gains 1, within 300 ms.

04 · RENDERING

One camera, two eyes, one final pass

A custom URP renderer feature draws both eyes into one side-by-side image, then a single final pass at full screen resolution does lens distortion, colour fringing correction, passthrough, lens effects and the late warp.



Frame budget (estimates, A16)

Main thread: ARKit, safety, kernel, blades, culling	**4.3 ms**
Render thread: ≈250 draws	**3.2 ms**
GPU: scene 3.0 · portals 1.5 · curved/fog 0.8 · compute 1.3 · final 1.0	**7.6 ms**
Hands on the Neural Engine (async)	**≈25 ms**
Budget at 60 fps	**16.7 ms**

GPU capped at 8 ms because phones sustain about half their peak clocks inside a headset. Spikes 0.15–0.17 replace these estimates.

Settings that make it fit

- Eye images 2 × 1216×1120 (≈0.87 of the panel); tiers step to 0.75 and 0.65.
- 4× MSAA kept in tile memory, never written out; 2× at the "serious" heat tier.
- Lens mask skips pixels outside the lens circle (≈30–40% fewer).
- ≤150k triangles per eye including what portals show; ≈250 draw calls (SRP Batcher).
- No HDR, shadows, SSAO or post stack; baked lighting and colour LUTs.
- Heat tier "serious": render at 30 fps, warp to 60.

Shaders and compute kernels we write

File	Kind	Does
AtlasUber.shader	lit material	Kit and scanned objects: baked light probes, colour grading folded in, outlines; SRP Batcher compatible
CurvedSpace.hlsl	include	One 4×4 isometry, the Klein divide (x/w, y/w, z/w), then the eye matrix; equidistant option per wing
GeodesicFog.hlsl	include	Fog by geodesic distance d = R·atanh(|x′|/w′); hard draw radius 3–4R
PortalStencil.shader	stencil	Portal frame writes a stencil value; the far side is drawn stencil-tested, clipped at the portal plane
Slicer4D.compute	compute	Tetrahedron ∩ hyperplane → triangles; prefix-sum compaction (deterministic); indirect draw arguments
HealpixSky.compute	compute	49,152 directions marched through distance-field volumes → risk texture; AND across the 10 phases
FinalComposite.shader	full screen	Distortion-mesh vertex shader with the late warp; fragment: colour-fringe fix, passthrough, FieldFit and colour LUTs, lens mask

Late warp, step by step. (1) Render both eyes with the pose predicted for photon time. (2) Just before the final pass, read the newest prediction: $\Delta R = R_{render}^{-1} \cdot R_{latest}$. (3) In the distortion-mesh vertex shader, rotate each vertex's eye-space direction by ΔR^{-1} and re-project it into that eye's half of the image. (4) Rotation only, no positional reprojection: it removes most rotational lag for almost no cost.

URP switches. HDR, depth texture, opaque texture, depth priming, post-processing and shadows off; Native RenderPass and SRP Batcher on; Render Graph on (compatibility mode off). Confirm in an Xcode GPU capture that MSAA colour and depth are never stored to or reloaded from memory.

05 · THE NON-EUCLIDEAN ENGINE

One interface, every kind of space

Positions are never stored as one global coordinate: a point is a **chart** (a room or cell) plus local coordinates, and charts are joined by **gluings** (portals, glued walls, branch cuts, time seams). Every geometry implements the same interface, so every technique works everywhere.

```
public interface IGeometry {                                  // Atlas.Kernel: no Unity scene types
    GeometryKind Kind { get; }       // Euclid, Hyperbolic, Spherical, Quotient, Branched, Portal, Space4, Time4
    double R { get; }                // curvature radius (infinite when flat)
    ChartPoint Exp(in ChartPoint p, in double3 v);        // take a step, reach, cast a ray
    double3    Log(in ChartPoint p, in ChartPoint q);     // direction and distance to q
    double     Distance(in ChartPoint p, in ChartPoint q);
    double3x3  Transport(in ChartPoint p, in double3 v);  // how a carried arrow turns
    int        Images(in ChartPoint p, Span<ChartPoint> dst);          // copies (glued, branched)
    bool       Crosses(in ChartPoint a, in ChartPoint b, out Gluing g); // wall, portal or cut?
}
```

Space	Stored as	Drawn with	Crossing logic
Euclidean + portals	Affine charts	Stencil portals (≤2 deep), clip plane in the shader	Step crosses a portal plane → apply its 4×4 transform
Glued (torus, Klein)	One cell + deck group	3×3 instanced copies; mirrored copies flip cull mode	Position mod 1.5 m; glide flips orientation
Hyperbolic $H^2 \times E$	Hyperboloid, double on the CPU	Klein chart in the vertex shader; geodesic fog	Camera moved by the exp map; re-anchor to the nearest cell every frame
Spherical / view sphere	Sphere coordinates	Gnomonic chart per hemisphere; HEALPix for the needle sky	Exp map; great circles
Branched (Shared House)	Chart + sheet index	Each sheet is a chart behind the door portal	Crossing the cut applies the sheet permutation
4D (Breath Room, toybox)	Tetrahedra in $\mathbb{R}^4$; per-case tube mesh	Compute shader slices tetrahedra; indirect draw	Slice normal = the cube's quaternion
Lens (FieldFit)	512^2 warp texture	Final pass lookup	Recomputed only when lens settings change

Precision rule. Double precision on the CPU, re-anchored every frame; floats on the GPU relative to the camera's cell. Airway trees store only parent-relative isometries (global hyperbolic coordinates overflow for deep trees).

Tested by maths. Unit tests check identities: exp∘log ≈ identity; a loop's turn = curvature × area; Riemann–Hurwitz on generated houses; a glide gluing flips orientation; CPU and GPU agree.

Formulas the kernel implements

Distance (H)	$d = R \cdot \operatorname{arcosh}(-\langle p,q \rangle / R^2)$, Minkowski product
Step (H exp map)	$\cosh(\lVert v \rVert/R) \cdot p + R \cdot \sinh(\lVert v \rVert/R) \cdot \hat{v}$ (S: cos, sin)
Klein chart	(x/w, y/w, z/w): straight lines stay straight
Fog distance	$d = R \cdot \operatorname{atanh}(\lvert x' \rvert / w')$, stable near the camera
Loop turn	$\theta = \lvert K \rvert \cdot A$ with $K = \pm 1/R^2$ (Gauss–Bonnet)
Airway curvature	$R = \ell / \ln b \rightarrow 1.5\ \text{m} / \ln 2 \approx 2.2\ \text{m}$
Circle of 1 m	$2\pi R \cdot \sinh(1/R)$ (H) · 2π (E) · $2\pi R \cdot \sin(1/R)$ (S)

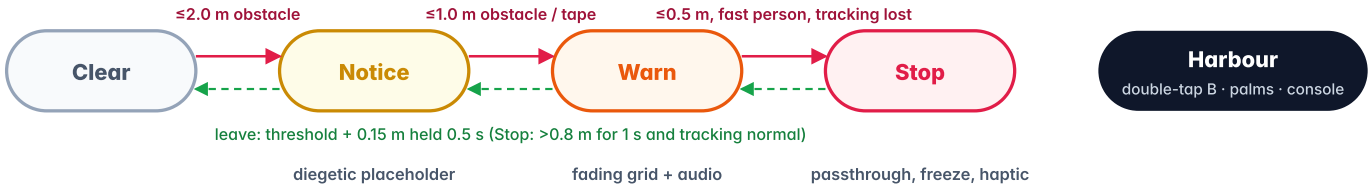
Code uses the *unit* hyperboloid and sphere and scales by R only when converting to metres.

```
public readonly struct ChartPoint {
  public readonly ChartId Chart;  // room or cell
  public readonly double4 Local;  // coords inside it
  public readonly int     Sheet;  // branched, else 0
}
public sealed class Gluing {      // world-graph edge
  public ChartId    From, To;
  public double4x4  Transform;    // affine / SO⁺(3,1)
  public GluingKind Kind;         // Portal, Wall, Cut,
                                  //   Hatch, TimeSeam
  public int[] SheetPermutation;  // branched (σ)
}
```

Inputs with twins, safety with hysteresis

Input	How	Rate	Twin / fallback
Walking	ARKit 6DoF pose + Core Motion prediction	60 Hz (+200 Hz IMU)	— (core)
Gaze reticle	Head direction; dwell 700 ms (600 ms on the needle sky)	every frame	Gamepad A
Gamepad	A = use / hold to carry, B = back, double-tap B = Safe Harbour	event	Headset lever tap
Hands	Vision hand pose on the Neural Engine; depth from LiDAR (≥0.32 m); One-Euro filter; Geometry Lens = two-hand frame held 300 ms	30 Hz	Trigger / gaze
Marker cube	ARKit image tracking (6 faces); pose fused from visible faces; quaternion kept continuous	60 Hz	Gamepad sticks
Voice	Heard by the narrator, pressed on the console (no phone speech recognition)	—	Console

Safety state machine (first in every frame)



Inputs: 7 depth sectors across ±35° (Burst job on the 256×192 depth image) · a 10 cm obstacle grid from the room mesh · distance to the taped outline (registered from the floor marker) · people from Vision at 5–10 Hz with closing speed · tracking state (limited for >0.5 s = lost).

Outputs: level + nearest-obstacle direction to rendering (placeholder, grid, passthrough), audio and haptics in the same frame. Paths that run near the tape by design may suppress the grid, never Stop. The ultra-wide lens mode pauses ARKit, so it is seated only.

States: enter, respond, leave

State	Enter when	Response	Leave when
Notice	obstacle ≤2.0 m	placeholder (tree, pillar) at the obstacle	>2.15 m for 0.5 s
Warn	≤1.0 m to an obstacle or the tape	fading grid + soft audio	>1.15 m for 0.5 s
Stop	≤0.5 m, fast-approaching person, or tracking lost	passthrough, locomotion frozen, haptic pulse	>0.8 m for 1 s with tracking normal, or operator
Harbour	double-tap B, palms down, or console	Safe Harbour	user or operator resumes

Hands pipeline (Swift plugin)

1. Copy what's needed from the ARFrame and release it at once (held frames stall the camera after ≈15–20 frames).
2. Vision hand pose at 30 Hz on the Neural Engine; latest-only queue; up to 2 hands.
3. Depth: median LiDAR depth over wrist + 4 knuckles; closer than 0.32 m, estimate from hand size.
4. Unproject with that frame's intrinsics; One-Euro filter; predict one frame ahead.
5. Draw a ghost hand and the camera-cone ring.

Privacy, enforced in code. Camera frames stay in RAM and are released the same frame; nothing is recorded or uploaded; depth and mesh are preferred over colour images; session IDs are anonymous. The team's own ARKit recordings are test data and never include judges.

From real rooms and objects to light, fast assets

Room scan (in the app)

1. RoomPlan runs inside the app's own AR session (no session switch).
2. Walls, doors, openings and furniture boxes are exported as JSON.
3. The room is rebuilt from our modular kit: clean walls; real doors become portal sockets; furniture becomes walls and touchable props.
4. The world map is saved for relocalisation; the floor marker re-anchors.
5. Scan the venue the day before and the morning of.

The judge's object (live)

1. A teammate takes 1–3 photos on the Galaxy S24 during framing.
2. Upload → background removal (rembg / SAM 3).
3. SPAR3D on the RTX 5070 (≈7–10.5 GB) → ≤10k triangles.
4. gltfpack (meshopt + KTX2) → "asset ready" → the phone places it as the airway cairn.
5. Fallback: a kit object by label, labelled as such.

Formats and budgets

Asset	Format	Budget
Room kit	GLB, meshopt, KTX2; sockets in glTF extras; 0.5 m grid	2048² trim sheets; ≤50k triangles per chunk
Objects	GLB + one 1024² KTX2 texture	≤10k triangles; 1–5 MB
Airway, vessels, lobes	GLB with two detail levels, split per lobe	≈115k triangles total
Breath Room tube	GLB, precomputed per case	≤10k triangles
Needle-sky volumes	3D KTX2, RGBA8 (vessels, fissures, bone, other)	≈9 MB per phase, ≈90 MB per case

```
case_<id>/
  manifest.json         versions, hashes, licences
  tree.json             airway graph (atlas.tree/1)
  airway_lod0.glb  airway_lod1.glb
  vessels.glb  lobes.glb  chambers.glb
  tube.glb  itv_table.json
  sky_phase0..9.ktx2  gtv_phase0..9.ktx2
  plancard_template.json
```

Loading without hitches. glTFast with a per-frame time budget, a material generator that maps to the ATLAS uber-shader, and pre-warmed shader variants: worst frame under 20 ms while a 5 MB asset loads (spike 0.25).

Object routes

Need	Route	Time (est.)
Speed (live)	S24 photos → rembg or SAM 3 → SPAR3D on the RTX 5070 → ≤10k triangles → gltfpack → phone	≈20–60 s
Fidelity	Object Capture on the iPhone → reconstruction on the M3 Max → USDZ → GLB	minutes
Best quality	TRELLIS.2 on the H200 (needs ≥24 GB); export at 5–20k faces	seconds + transfer
Fallback	Pre-generated kit object chosen by label, shown as "pre-generated"	instant

Compression. Textures are KTX2 with Basis Universal: ETC1S for colour (small), UASTC for normal maps (quality), transcoded to ASTC on the iPhones at load. Geometry is meshopt-compressed by gltfpack.

Coherence. Raw photo lighting is never shown: keep the colour, re-light with the region's baked probes and LUT, and animate the scan turning into geometry so the value is visibly what the space does to it.

08 · ROOM KIT

The modular kit every room is rebuilt from

Scanned rooms and generated chambers are both assembled from one kit: 0.5 m grid, sockets stored in glTF extras, one 2048² trim sheet per region, KTX2 textures and meshopt geometry. Triangle counts are LOD0 budgets.

Module	Variants	Sockets / metadata	Tris	Used by
Wall panel	0.5 m, 1.0 m; plain, window, trim	`wall` (2.6 m high)	≤600	Hub rebuild, chambers
Corner	inner, outer	`corner`	≤400	Hub rebuild
Door frame	single, double, free-standing	`portal`: stencil plane, clip plane, transform	≤800	Hub, Post Circling, chambers
Floor tile	0.5 m, 1.0 m; plain, marked X	`tile` (walkable flag)	≤50	Every room
Furniture proxy	box, table, chair, couch, shelf, scaled from RoomPlan boxes	`prop`, `haptic` (touchable)	≤1,000	Hub rebuild
Fork chamber	1 in / 2 out, 1 in / 3 out	`valve_door` ×N, `beacon`	≤3,000	Airway
Corridor	straight 1.0–2.0 m; curved for R = 2.2 m (left, right)	`corridor_in`, `corridor_out`	≤1,500	Airway
Valve door	open / close animation	`valve`	≤500	Airway
Cairn plinth	—	`cairn` (object anchor)	≤300	Airway
Glued cell shell	1.5 m T^2 cell; T^3 variant 2.6 m high	`deck` (instanced copies)	≤4,000	Glued Room, Breath Room
Gating flag	pair	`flag`	≤200	Breath Room
Branch post	post with ring marker	`branch_post` (sheet permutation)	≤400	Post Circling
Beacon, sign, caption chip	—	`ui_anchor`	≤200	All
Safety placeholder	tree, pillar, rock	`obstacle`	≤1,500	Safety "Notice" tier

Placement rule. Every room is occupied tiles plus door sockets on tile edges. A placement is valid only if all its tiles lie inside the eroded free floor and every door socket faces at least one free tile. Rooms seen at the same time never overlap; hidden ones may.

Coherence. Each region gets its LUT, fog colour, trim sheet and audio bed from the style cascade (region → room → props), so scanned and generated rooms read as one designed world.

From a RoomPlan scan to kit

1 · Scan
RoomPlan JSON: walls, doors, windows, openings, objects with category and size

2 · Snap
Walls snapped to the 0.5 m grid where they fall within tolerance

3 · Sockets
Doors become portal sockets; windows become window panels

4 · Props
Each object box becomes the proxy of its category, scaled to fit

5 · Floor
Free floor eroded by 0.3–0.5 m and cut into tiles

6 · Style
Region LUT, trim sheet and audio bed applied

09 · NETWORKING AND THE PC SIDE

A simple server the experience never waits on

Port / path	Protocol	Purpose
8765 /ws	WebSocket	Control for phone, partner, console, spectator
8766	UDP	Head pose 30–60 Hz (latest wins)
/assets/{hash}	HTTP	Content-addressed assets
/api/case/{id}/build	HTTP	Roll a Lung: complete, lay out, assemble
/api/object	HTTP	Judge's photos → mesh
/api/logs	HTTP	Session logs at exit
/console, /dashboard	web	Operator console, Knife Map dashboard

Discovery: QR code or mDNS (`_atlas._tcp`). **Development:** USB tunnels (`iproxy`, `adb reverse`). **Venue:** own router, fixed 5 GHz channel, AirDrop/Handoff off.

Link health (phone side)

Connected	heartbeat within 500 ms
Degraded	0.5–3 s (2 misses in, 2 s healthy out): ignore stale plans, ease gains to 1, freeze world edits
Offline	>3 s: cached world, local planner, reconnect with backoff
Resync	clock sync + world-version delta

Messages (one schema, MessagePack)

Hello/Welcome · Ping/Pong (clock sync) · Pose · Event · KnifeState · StateSync · PlanUpdate · WorldDelta · CaseBuild · AssetReady/Ack · OperatorCmd · HouseState · HandOver · HazardPin · Heartbeat · Error/Bye. One YAML file generates the C# and Python types; every message carries a protocol version.

On the PC

Server

One asyncio process: link, UDP, assets, case and object jobs, console commands, logs (SQLite → Parquet). One GPU worker process owns the RTX 5070.

Console + dashboard

A web app on any tablet: fps and frame times, heat, tracking, round trip, battery, kill switches, blade queue, panic → passthrough, re-anchor, comfort branch.

Spectator

The same Unity project built for Windows; fed replicated state (chart, sheet, cell, transforms) at 10–30 Hz plus pose; renders the mirror and chart strip.

What goes over the wire

```
{"type":"Hello","proto":[1,0],"device":"iPhone15ProMax",
 "platform":"ios","app":"0.4.0","role":"phone"}
{"type":"Pose","seq":10231,"t_us":8812345678,
 "pos":[0.41,1.62,-0.88],"rot":[0.01,0.71,0.0,0.70],
 "chart":"lung.airway.n37","sheet":0,"cell":"0,0",
 "track":"normal"}
{"type":"CaseBuild","case":"C07","status":"ready","progress":1.0,
 "asset_ids":["tree_C07","chambers_C07"]}
{"type":"OperatorCmd","cmd":"panic","args":{}}
```

Conventions

Unity world space (left-handed, Y up, metres) · quaternions as [x, y, z, w] · timestamps in phone microseconds, converted on the PC with the clock offset ((t2 − t1) + (t3 − t4)) / 2 · every connection opens with `Hello` and a protocol version · shown here as JSON, sent as string-keyed MessagePack.

Roll a Lung, end to end (≤10 s)

1 · Pick
The operator taps the judge's case card on the console → POST `/api/case/{id}/build`

2 · Load
The PC reads that case's pre-segmented masks and skeleton from its cache

3 · Complete
Small airways grown by the space-filling model, tagged "model"

4 · Lay out
Tree placed in H^2 ($R \approx 2.2$ m) → `tree.json`

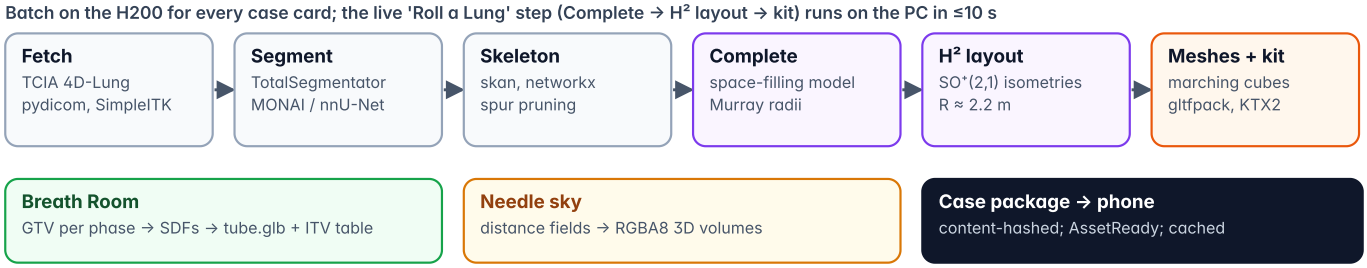
5 · Assemble
Kit chambers and corridors → `chambers.glb`, tile metadata

6 · Deliver
`CaseBuild` progress → `AssetReady` → phone fetches by hash, loads in budget

10 · MEDICAL DATA PIPELINE

From a public CT to a walkable case

Each case card is a TCIA 4D-Lung patient, prepared in advance on the H200. At the booth, the judge's pick triggers the live part: completing the small airways, laying the tree out in hyperbolic space and assembling the corridors, in about 10 seconds.



Command	Libraries	Output
atlas case fetch / convert	TCIA API, pydicom, SimpleITK, dcm2niix	CT per breathing phase
atlas case segment	TotalSegmentator; MONAI / nnU-Net airway model (benchmarked)	Lobes, vessels, airways, bone
atlas case gtv	rt-utils, SimpleITK	Tumour masks for 10 phases
atlas case skeleton	skan, networkx (spur pruning, spanning tree)	Branch graph with radii
atlas case complete	numpy, scipy (space-filling growth, Murray radii)	Branches tagged "model"
atlas case layout	numpy (hyperbolic isometries, R ≈ 2.2 m)	tree.json
atlas case mesh / chambers	scikit-image, trimesh, gltfpack, kit modules	GLBs
atlas case tube / sky / pack	SimpleITK, scipy, toktx	Tube, ITV table, sky volumes, package

H200 batch jobs

Segment every case (array job) · complete + lay out · tubes and sky volumes · the greedy-routing study (Euclidean vs hyperbolic branch choice on 1,000 targets per tree) · TRELLIS.2 kit props. Run in Apptainer containers under Slurm; results synced to the PC nightly.

Watch-outs. Airway segmentation is reported to stop after 1–2 branchings, so the model choice is a Phase 0 spike. Model-generated airways are always labelled. ATM'22 airway labels can't be redistributed. GPL tools stay out of shipped code.

The tree file the phone walks (`atlas.tree/1`)

```
{"schema":"atlas.tree/1","case":"C07","R_m":2.2,
 "nodes":[{"id":0,"parent":-1,"gen":0,"label":"trachea",
           "source":"segmented","len_mm":118,"radius_mm":8.9,
           "h2":{"angle_rad":0.0,"len_m":2.0,
                 "parent_iso":[[1,0,0],[0,1,0],[0,0,1]]}}, …],
 "target":{"node":57,"lesion":"L1","gap_mm":6.2},
 "phases":{"count":10,"gtv_sdf":"gtv_phase{0..9}.r8.ktx2"}}
```

`source` is "segmented" or "model"; `parent_iso` is the SO⁺(2,1) isometry from the parent chamber, so deep trees never overflow.

Triangle budget (visible)

Airway mesh (split per lobe)	**≈40k**
Vessels	**≈30k**
Lobes (5 × 3k)	**15k**
Corridors and chambers	**≈20k**
Tube, overlays, cairn	**≈10k**
Total, of 150k per eye	**≈115k**

11 · BLADE BUILD SHEET

What each live blade needs

Blade	Build components	Data	Key algorithms	Done when
Hub + Bend Your Room	Kit-rebuilt room, κ-family vertex shader, clutch-gated dial, carried arrow, Geodesic Ray	Venue RoomPlan scan	Constant-curvature family E ↔ H ↔ S; turn = curvature × virtual loop area	Smooth at 60 fps on phone B; turn within 5° of prediction
Breath Room	Glued-cell engine with x as time; floor flags; end-on ITV view; Slice Tilt (7 precomputed tilted slices)	Tube mesh + ITV table per case	Position mod 1.5 m → phase; ITV = union over the flagged phases	Tube visible across ≥3 copies at 60 fps; ITV matches table
Classic Glued Room	3×3 instanced copies, mirrored draw, self-avatar, Glue Die (cube)	Kit room	Deck translations; glide reflection; minimum image	No visible seam; correct mirror image
Lung · airway	Chambers, corridors, valve doors, tile solver, endoluminal window, Two Compasses, cairn, pointing probe	tree.json, airway GLBs	H^2 layout; exp-map walking; tile placement; greedy rule from the study	4 forks in ≤75 s for ≥90% of naive users
Lung · needle sky	Sphere-traced risk compute, look-to-aim, needle readouts, great-circle check, plan card	Sky volumes	HEALPix equal-area grid; robust windows = safe in all 10 phases	Current phase <100 ms; plan card validates
Post Circling	Two pre-scanned sheets, free-standing door, recorded avatar	Two room scans	Sheet permutation on crossing the cut; round one post = switch	Works with no network
FieldFit (choice)	Blind-spot finder, tunnel field, power-law lens, warp texture, Lens-6DoF	Visual fields (UWHVF)	Schwarz bound; power-law minimax lens	Blind spot in ≤30 s for ≥80%; lens <1 ms
House Call (choice)	S24 partner client, LiveKit voice, avatar, hazard pins, chair-stand timer	Judge's room scan	Marker-based alignment; head-height chair-stand detection	Two devices on the LAN; recorded fallback

The 10-minute run, seen from the engine

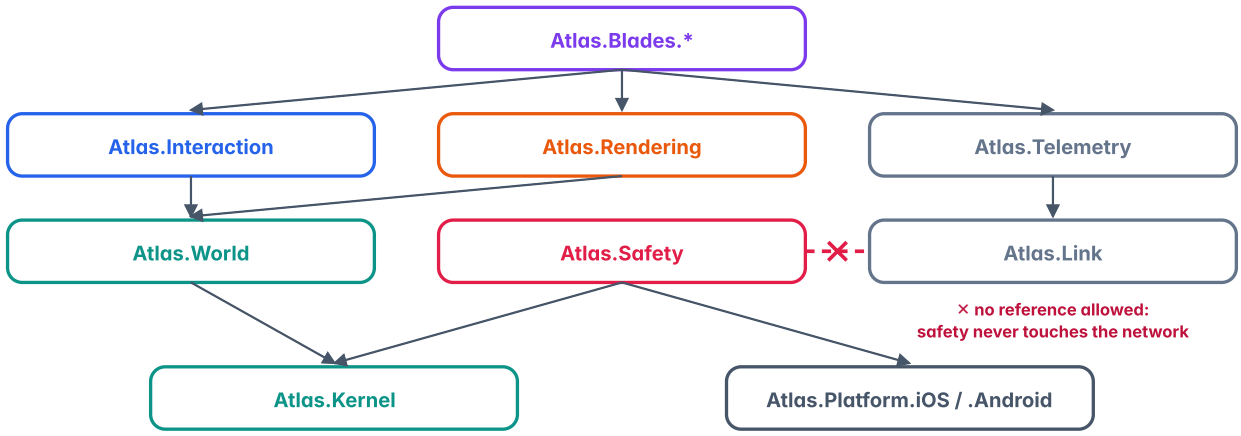
Time	Segment	Space	What runs or loads	If it fails
0:00–0:40	Frame	—	Roll a Lung build on the PC; S24 photos → SPAR3D; baseline comfort score	Default case; kit object
0:40–1:20	Onboarding	passthrough	Fit, floor-marker re-anchor on the X, gamepad check, hands check	Pad mode
1:20–1:50	Bend Your Room	E → H / S	Hub from the venue scan; curvature dial; carried arrow	Day-before scan
1:50–2:35	Breath Room	$\mathbb{R}^3 \times S^1$	Glued cells with x as time; `tube.glb`, ITV table, tilted slices	Classic Glued Room
2:35–3:50	Airway	H^2	`tree.json`, chambers, tile solver, Two Compasses, cairn, pointing probe	Cached default tree
3:50–4:20	Needle sky	S^2	Sky volumes, HEALPix compute, great-circle arcs, plan card	Precomputed sky
4:20–4:40	Post Circling	branched	Two pre-scanned sheets, recorded avatar	Skip (comfort)
4:40–5:25	Judge's choice	sphere of view / E	FieldFit warp texture, or House Call (S24 + LiveKit)	Recorded partner
5:25–6:10	Exit + reveal	passthrough	Comfort scores; logs to `/api/logs`; dashboard	Phone's log buffer

Preload rule. The next segment's package loads while the current one runs (glTFast with a frame-time budget), and the one before it is unloaded, so peak memory stays under 1.8 GB on phone B. Nothing on this path waits on the network once the case is built.

12 · CODE ORGANISATION

Assemblies with enforced boundaries

The Unity project is split into assemblies whose references only point downwards. The build itself stops the safety code from depending on networking, and keeps the geometry kernel free of engine objects so it can be tested on its own.




```
atlas/
  phone-unity/            Unity 6.3 project (iOS, Android, Windows spectator)
    Assets/Atlas/Kernel/      geometries, double maths, identities
    Assets/Atlas/World/       charts, gluings, tile solver, case loader
    Assets/Atlas/Rendering/   stereo feature, portals, shaders, final pass
    Assets/Atlas/Interaction/ techniques, twins, gestures
    Assets/Atlas/Safety/      sectors, grid, state machine
    Assets/Atlas/Link/        WS/UDP/HTTP, messages, clock sync
    Assets/Atlas/Telemetry/   logger, KnifeState, StateSync, probes
    Assets/Atlas/Blades/      Hub, GluedRoom, BreathRoom, Lung, ...
    Assets/Plugins/iOS/       Swift: Vision, RoomPlan, Core Motion, ...
  server/                 FastAPI app, workers, web console
  pipelines/              atlas CLI (medical, objects, rooms, fieldfit)
  hpc/                    Slurm + Apptainer jobs
  specs/                  messages.yaml, tree/plancard schemas
  ci/                     GitHub Actions, xcodebuild/devicectl, perf scripts
  docs/                   maths notes, licences, demo runbook
```

Native iOS plugin: C entry points

`atlas_vision_start / _poll`	Hand pose on the Neural Engine, latest only
`atlas_humans_poll`	People boxes at 5–10 Hz (safety)
`atlas_motion_start / _predict`	Core Motion 200 Hz; pose at photon time
`atlas_roomplan_begin / _end`	RoomPlan on the app's AR session; JSON + USDZ
`atlas_worldmap_save / _load`	Relocalisation
`atlas_thermal_state`	Heat state for the tiers

Swift with `@_cdecl`; C# uses `DllImport("__Internal")`; ARKit objects come from AR Foundation's native pointers.

Interfaces that tie it together

IGeometry (kernel) · **Gluing** and **ChartPoint** (world) · **ITechnique** (logs its design-space code and an equation card) · **IBlade** (load, enter, tick, exit, snapshot for the spectator) · **ISafetyMonitor** (level + nearest obstacle, never awaits).

Engineering standards

No per-frame allocations in render or safety loops · double-precision geometry on the CPU · every technique registers its code · deterministic replay of logged inputs · licence ledger (MIT/Apache by default; GPL, non-commercial and gated items flagged).

13 · SET UP IN A DAY

Versions, packages and the first build

Pin these versions

Unity	6000.3 LTS (Unity 6.3), exact patch pinned
URP	17.x bundled with 6000.3; Render Graph on
AR Foundation	6.x for 6000.3; ARKit + ARCore plugins
iOS target	17.0; Xcode supported by Unity 6000.3
Cardboard SDK	v1.35.0, C API, optics only
Python	3.11 (server and pipelines)
PyTorch	≥2.7 with CUDA 12.8 on the RTX 5070

Unity packages

URP · AR Foundation, ARKit, ARCore · XR Management (no display provider) · Input System · Burst, Collections, Mathematics · glTFast + meshopt + KTX · Test Framework + Graphics Test Framework · Profiling Core · **third-party:** UniTask, MessagePack v3, Apple Unity plugins, LiveKit, UniVRM, uLipSync, Steam Audio (optional), Sentry.

Native plugin and plist

Swift sources in `Plugins/iOS`, C entry points for Vision, people detection, Core Motion prediction, RoomPlan, world map and thermal. A post-build script sets camera, local-network, Bonjour and microphone strings, landscape only, and the increased-memory entitlement if the free team allows it.

Day-1 checklist

1. Create the Unity 6000.3 URP project; add the packages; set up Git LFS for GLB, KTX2, PNG and FBX.
2. Player settings: IL2CPP, ARM64, Metal, iOS 17.0, landscape left, Frame Timing Stats on, 60 fps target.
3. XR Management: ARKit on (iOS), ARCore on (Android), no stereo display provider.
4. Sign in to Xcode with the free Apple account; in Unity set Signing Team ID to the Personal Team and tick Automatically Sign; fix the bundle ID. Turn on Developer Mode on both phones.
5. Build an empty AR scene to both iPhones; confirm tracking and the depth image; try the memory capability (spike 0.26).
6. Add the Cardboard library; scan the headset's QR; check straight lines stay straight at the edges.
7. Start the FastAPI server; connect over USB, then Wi-Fi; see pose packets on the console.
8. Wire CI: commit → Unity build → `xcodebuild` (free team) → `devicectl` install on both phones in ≤20 min.

Accounts: a free Apple account signed in to Xcode (Personal Team; no enrolment; see Signing and installing), GitHub with a self-hosted runner on the M3 Max, Unity licence, H200 cluster access (confirm batch vs interactive).

On the PC: CUDA 12.8 driver, PyTorch cu128, gltfpack, toktx and dcm2niix on the path; one GPU worker process owns the card.

Info.plist, entitlements, build settings

`NSCameraUsageDescription`	Tracking, hand pose, passthrough
`NSLocalNetworkUsageDescription`	Link to the ATLAS PC on the LAN
`NSBonjourServices`	`_atlas._tcp` (discovery)
`NSMicrophoneUsageDescription`	House Call voice only
Screen	`UIRequiresFullScreen`; landscape left only
Entitlement	`com.apple.developer.kernel.increased-memory-limit`, if the free team accepts it (spike 0.26)
Build	IL2CPP, ARM64, Metal only, Swift version set, Bitcode off, VSync 1, stripping Low

Python and the H200

Server: fastapi, uvicorn, msgpack, pydantic 2, numpy, qrcode, zeroconf.
Pipelines: numpy, scipy, scikit-image, SimpleITK, pydicom, nibabel, rt-utils, TotalSegmentator, monai, skan, networkx, trimesh, pygltflib, astropy-healpix.
Locking: one `pyproject.toml` each, lock files committed (uv or pip-tools).
H200: an Apptainer image (`atlas-pipelines.sif`) built from the same pyproject; Slurm array jobs in `hpc/`; outputs in `/project/atlas/`, synced to the PC's cache.

Fast inner loop. Work that doesn't need tracking runs in the Unity Editor with AR Foundation's XR Simulation environments. Kernel maths runs as EditMode tests and in the PC tools, no device needed. On the phone, stay on USB with `iproxy` so the console shows frame times live, and replay recorded ARKit walks (team-only) to repeat tracking bugs.

Things that bite on day 1. The Local Network prompt must be accepted again after a reinstall (spike 0.21). Free-account builds stop launching 7 days after install. Retained ARFrames stall the camera. PyTorch on the RTX 5070 needs cu128 wheels; older repos with custom CUDA kernels may need rebuilding.

A free Xcode account, and how builds reach the phones

We sign with a free Apple account in Xcode, which Xcode calls a **Personal Team**: no enrolment, no fee, no review wait. Its limits are small for a two-phone project, and the rules below keep them out of the way. The paid Apple Developer Program is the fallback, not the plan (decision D19).

Build → phone

1 · Unity
iOS build with Signing Team ID = the Personal Team, Automatically Sign on

2 · Sign
`xcodebuild` with `-allowProvisioningUpdates`, using the account signed in to Xcode

3 · Install
`xcrun devicectl` on phone A and phone B, plugged into the M3 Max

4 · First run
Developer Mode on; trust the developer in VPN & Device Management

5 · Check
Launch each phone once with Wi-Fi off

6 · Fallback
Open the project in Xcode and press Run

```
xcodebuild -project Unity-iPhone.xcodeproj -scheme Unity-iPhone -configuration Release \
  -destination 'generic/platform=iOS' -derivedDataPath build -allowProvisioningUpdates build
xcrun devicectl list devices                                  # phone IDs
xcrun devicectl device install app --device <udid> build/Build/Products/Release-iphoneos/<Product>.app
# flags verified in spike 0.24
```

Free-account limits and our rules

Limit	Rule
Builds stop launching 7 days after install	Every rebuild renews it. Before an event, install on both phones ≤24 h before day 1; the launch check before each session catches an expired build
3 registered devices	Only phone A and phone B on the owner's account; teammates test on their own phones with their own free accounts
3 apps per device	ATLAS plus at most one test app
10 new app IDs per 7 days	One fixed bundle ID, `com.<owner>.atlas`, for the whole project; no throwaway projects on this account
No TestFlight or developer portal	Installs go from the shared Mac over USB; Xcode manages certificates and profiles automatically
Some capabilities not offered	The only one we request is Increased Memory Limit. Spike 0.26 checks it; if refused, drop it and pass spike 0.22 (≤1.8 GB on phone B) without it

Event procedure

Freeze the golden build. The evening before day 1, install it on both phones and launch each once with Wi-Fi off. Bring the Mac and both cables: a reinstall takes about 5 minutes, and phone B covers while it runs. A two-day event needs no reinstall in between.

When we'd pay (US$99/yr, approval usually ≈48 h): spike 0.26 fails and the app can't hold 1.8 GB without the capability, more devices or testers are needed, or manual installs become a bottleneck. Enrol as an individual ≥2 weeks before the event. Switching takes 15–30 min: change the team, delete the free-signed copy from each phone, add the capability, rebuild. Steps: spec Appendix E.

How we keep it fast and keep it working

Tests

- Kernel identities in double precision (exp∘log, loop turn = curvature × area, Riemann–Hurwitz, glide flips orientation).
- CPU vs GPU agreement for the Klein, HEALPix and slicer kernels (astropy-healpix as reference).
- Golden images per blade (Unity Graphics Test Framework).
- Message round-trip tests from the single schema.
- Pipeline golden outputs on one reference case (pytest).
- ARKit session recordings to replay tracking (team-only test data).

CI/CD

- Self-hosted GitHub Actions runner on the M3 Max; Unity batch mode with a cached Library folder.
- Post-build script sets Swift settings, entitlements and plist keys.
- `xcodebuild` signs with the free Personal Team; `devicectl` installs on both phones on the Mac's USB; backup: Run from Xcode (see Signing and installing).
- Freeze a golden demo build and the pipeline before the event.

Performance and stability

- Nightly replay on phone B: CPU/GPU p99, missed frames, memory, heat; fail on >10% regression.
- Instruments (Metal System Trace, Allocations); a weekly GPU capture per blade.
- Memory ≤1.8 GB on phone B; textures ≤300 MB, meshes ≤150 MB, volumes ≤100 MB per blade.
- Crash reporting with an offline queue.

Kill switches and demo day

- On the phone (gamepad chord, works offline) and on the console: hands, cube, meshing, people detector, live vs cached sky, portal depth, glued copies, MSAA, render scale, network.
- Guided Access, orientation lock, Do Not Disturb, AirDrop and Handoff off.
- Videos play from the PC; logs encrypted and pseudonymous.

Heat tiers (both phones use phone B's tier)

Heat state	Settings
Nominal / fair	Full quality, 60 fps, eye scale 0.87, 4× MSAA
Serious	Render at 30 fps and warp to 60; ARKit stays at 60 Hz; lower detail and scale; 2× MSAA; hands ≤15 Hz
Critical	Minimal scene and passthrough
Swap rule	Swap phones between judges at heat state "fair" or after 30 min; in a session, drop a tier instead

Commit to both phones (≤20 min)

1 · Push
GitHub Actions on the self-hosted M3 Max runner

2 · Unity
Batch-mode iOS build with a cached Library folder; EditMode tests

3 · Post-build
Script sets plist keys, entitlements and Swift settings

4 · Sign
`xcodebuild` signs with the free Personal Team

5 · Install
`xcrun devicectl` on both iPhones on USB; backup: Xcode Run

6 · Nightly
Replay on phone B; fail on a >10% p99 regression

Numbers we measure and publish

fps p50 and 1%-low per blade, both phones, after a 30-min heat soak

Motion-to-photon latency (240 fps camera or photodiode)

Tracking drift per metre walked

Obstacle-warning latency (target <200 ms)

Numerical drift of the hyperboloid after N steps

Hand-tracking visible fraction and latency; cube jitter

Scan-to-world time for rooms and objects

Asset load-to-visible time

Measure first, then build the story

Phase	Build	Done when
0 · Spikes (≈2 weeks)	24 platform benchmarks on phone B in the headset (below)	Every spike has a number and a decision
1 · Kernel, hub, safety, tooling	Kernel (E, T, P), world model, one-camera stereo + final pass, portals, in-app room scan + kit, relocalisation, safety, gamepad and gaze, Geodesic Ray, classic Glued Room, logging, schema + server, console, spectator, Knife Map, CI, kill switches	Glued Room at 60 fps on phone B; Safe Harbour <300 ms; commit-to-device ≤20 min
2 · Story path core	Bend Your Room, Breath Room, Lung pipeline, Roll a Lung, Two Compasses, endoluminal view, needle sky, plan card, pointing probe, object pipeline, greedy study	4 forks in ≤75 s; Roll a Lung ≤10 s; sky phase <100 ms
3 · Post Circling + choices	Solo Post Circling, FieldFit (Lens-6DoF), S24 partner client + voice	Both choices run in the 10-min show
4 · Hands, polish, show	Hands + twins, Geometry Lens, cube, style and audio, dashboard reveal, heat tiers, videos, rehearsals	All rehearsal targets met

Phase 0 spikes

0.1 Stereo on both phones: GPU ≤8 ms at 150k tris/eye
0.2 ARKit through the window: drift <2% over 20 m
0.3 Hand reach; Geometry Lens detected ≥90%
0.4 LiDAR hand depth valid at ≥35 cm
0.5 Cube jitter, latency, loss
0.6 Portals in stereo <1.5 ms at depth 2
0.7 Klein vs equidistant stereo per curved wing
0.8 Closed-headset heat soak: 20 min before a tier drop
0.9 Wi-Fi round trip p99 <30 ms
0.10 Lens-mode switch (research mode)
0.11 RTX 5070 model timings
0.12 H200 access policy
0.13 Viewer profile on both phones
0.15 One camera vs two: CPU ≤7 ms
0.16 Motion-to-photon ≤30 ms with prediction
0.17 Vision on the Neural Engine: no GPU cost
0.18 RoomPlan inside the app session; relocalise <3 cm
0.19 Airway depth and greedy-routing study
0.20 Needle sky, 10 phases <300 ms, ≤100 MB
0.21 3-device Wi-Fi loss <1%
0.22 Memory: 20 blade cycles, no crash
0.24 CI commit-to-device ≤20 min
0.25 Asset load: worst frame <20 ms
0.26 Free Xcode signing on both phones; memory capability

Which spike settles which decision

Decision	Spikes	Default until measured
Stereo path (D5)	0.1, 0.15	One camera, side-by-side; two-camera rig as fallback
Curved stereo per wing (D9)	0.7	Klein per eye; equidistant if depth feels wrong
Latency method (D16)	0.16	Core Motion prediction + late rotational warp
Airway model, R and corridor rule	0.19	TotalSegmentator + model completion; R ≈ 2.2 m
Needle sky on the phone	0.20	Live current phase, others precomputed
Budgets (GPU, memory, heat)	0.1, 0.8, 0.22	8 ms GPU, 1.8 GB, 20 min before a tier drop
Geometry strategy (D15)	0.14, after Phase 2	Portals + curved wings
iOS signing (D19)	0.26, 0.24	Free Apple account (Xcode Personal Team)

Cut first, in order: the Cardboard XR plugin spike → ultra-wide lens mode → live House Call → Steam Audio → live object meshes → curved-only mode → the judge's-choice slot. **Never cut:** safety, Safe Harbour, the story's geometry moments, the plan card, the dashboard.

The make-or-break tests, step by step

Write the pass mark and plan B down before running each test. Record results in `docs/phase0.md` with the date, phone, build and numbers.

0.1 + 0.15 · Stereo performance on phone B

Set-up: closed headset, room temperature; scene A empty, scene B 150k tris per eye + 2 portals.
Steps: 20 min per scene, logging CPU/GPU ms each second; once with one camera, once with two.
Pass: GPU ≤8 ms and main + render ≤7 ms at p99 after 20 min; ≤1 missed vsync/min.
Plan B: eye scale 0.75, MSAA 2×, portal depth 1; two cameras if they win.

0.2 · ARKit through the camera window

Set-up: closed headset, textured mat, even light; a taped 20 m loop.
Steps: walk it at normal pace and stop on the X; repeat in dimmer light.
Pass: drift under 2% of distance; no tracking loss in normal light.
Plan B: the other headset, more floor texture, re-anchor on the X before every blade.

0.7 · Comfort in curved space

Set-up: an H^2×E corridor at R = 2.2 m in both stereo modes; 5 first-timers.
Steps: comfort score at entry; 2 min per mode, order swapped between people; score after each; "did the depth look right?"
Pass: median rise ≤2; nobody stops; one mode judged right by 4 of 5.
Plan B: R of 3–5 m, a shorter airway segment, a vignette.

0.19 · Airway depth on the case cards

Set-up: 5 4D-Lung cases; TotalSegmentator vs a MONAI / nnU-Net airway model on the H200.
Steps: segment, skeletonise, count generations to each lesion; run completion and the greedy-routing study.
Pass: ≥6 generations segmented or completed for a 4-fork route.
Plan B: a curated default case, labelled completion, a 3-fork route.

0.26 · Free-account signing

Set-up: the owner's free Apple account in Xcode; both phones paired; fixed bundle ID.
Steps: add Increased Memory Limit; install on both phones with `xcodebuild` + `devicectl`; note the expiry date; reinstall after 6 days.
Pass: both phones install; the capability is accepted, or spike 0.22 passes without it.
Plan B: the paid program (spec Appendix E), at least 2 weeks before the event.

Order for week 1: 0.1 + 0.15 and 0.2 first (if the phone can't hold 60 fps in the headset, nothing else matters), then 0.26 and 0.7, with 0.19 running on the H200 in parallel. Every other spike follows in week 2.

Results log (`docs/phase0.md`)

Spike	Date	Phone / build	Key numbers	Pass?	Decision or plan B taken
0.1 + 0.15	____	B / ____	GPU p99 ___ ms · CPU p99 ___ ms · missed vsync ___/min	__	____
0.2	____	B / ____	drift ___% over ___ m · drops ___	__	____
0.7	____	B / ____	median comfort rise ___ · depth right ___/5 · stops ___	__	____
0.19	____	H200 / ____	segmented gen ___ · completed gen ___ · greedy H ___% vs E ___%	__	____
0.26	____	A + B / ____	capability accepted? ___ · install ___ min · expires ____	__	____

What could break, and the plan for it

Risk	Level	Mitigation
Heat throttling over a demo day	high	GPU budget at sustained clocks; heat tiers; meshing off after mapping; vented headset; fan; swap rule
No single-pass stereo on iOS (CPU cost)	high	One camera draws both eyes; portals as extra draw lists; spike 0.15
Airway segmentation too shallow	high	Benchmark models (spike 0.19); complete distal airways by model, labelled; curated default case
Hands not seen (narrow camera cone)	high	Hands always optional; targets below eye level at 40–55 cm; good light
Motion-to-photon too high	medium	Core Motion prediction + late warp; spike 0.16
Tracking loss (dark hall, plain floor)	medium	Textured mat, even light, saved world map, re-anchor on the X, passthrough on loss
Venue Wi-Fi interference	medium	Own router, fixed channel; everything the user feels runs offline
Memory pressure on the 6 GB phone	medium	Budgets per blade; memory entitlement if the free team allows it (0.26); spike 0.22
Headset doesn't fit both phones	medium	Buy two candidates now; test both camera bumps
Licences (GPL, non-commercial, gated)	medium	Licence ledger; GPL tools only internal; check SPAR3D and SAM 3 terms
Free build expires	medium	Install ≤24 h before; launch check; Mac on site; spike 0.26; paid fallback

Open engineering questions

1. Which stereo path wins on phone B (one camera vs two)?

2. Klein or equidistant stereo for each curved wing?

3. Which airway model reaches ≥6 generations on 4D-Lung?

4. Does RoomPlan share the app's AR session cleanly on both phones?

5. H200: batch only, or interactive from the venue?

6. Which headset fits both camera bumps and glasses?

7. Is there a 4th person for the live House Call option?

8. Which ≈20 4D-Lung patients go into the case-card deck?

Fallbacks on the day

Failure	Detected by	Immediate action	If it persists
Tracking lost or drifting	tracking state, jump	Fade to passthrough; back to the X; re-anchor	Phone B; then video
fps <45 or heat "serious"	console tile	Drop a tier	Swap phones between judges
Wi-Fi down	round-trip tile	Blades run standalone; dashboard catches up from the phone's buffer	AirPlay or video
AI service or H200 unreachable	service tile	Pre-baked assets on the phone, labelled "pre-generated"	—
Hands fail (light, sleeves)	hand-confidence tile	Pad mode: every gesture has a twin	Narrator drives the console
Person enters the square	LiDAR, people detector, spotter	Automatic passthrough; spotter says stop	—
Judge uncomfortable	comfort score, verbal	Stationary blade or end; chair and water	Offer the videos
Total failure	—	3-minute video + dashboard from earlier visitors	—

Docs and repositories to keep open

Unity and XR

Unity 6.3 LTS · AR Foundation · ARKit XR Plugin · single-pass stereo platforms · glTFast · MessagePack-CSharp · UniTask · Apple Unity plugins

Optics

Cardboard SDK · cardboard.h · releases · viewer profiles

Apple platform

ARKit scene depth · world maps · RoomPlan · RoomPlan with a custom AR session (WWDC23) · Vision hand pose · local network privacy · ARKit recording and replay · free vs paid accounts · Developer Mode

Assets, audio, multi-user

meshoptimizer / gltfpack · KTX-Software · Steam Audio · LiveKit Unity SDK · UniVRM · uLipSync · UXF

Medical and AI

TotalSegmentator · TCIA 4D-Lung · airway depth report · SPAR3D · TRELLIS.2 · SAM 3 · UWHVF

Non-Euclidean rendering

Non-Euclidean VR I · II · HyperEngine · SebLague/Portals · Hypermine · portals in VR (stencil vs render texture)

Real, pre-baked or mocked (fill in honestly before the event)

Item	Live	Pre-baked
Venue room scan	RoomPlan the morning of	Day-before backup
Case cards: segmentation, sky volumes, tubes, ITV tables	—	✓ on the H200 / PC
Roll a Lung (completion, layout, kit, tiles)	✓ ≤10 s	Segmentation of the case cards
Judge's object (S24 photos → SPAR3D)	✓ ≈20–60 s	Kit object fallback
Post Circling partner	—	Recorded avatar
House Call partner (option)	✓ two devices	Recorded track
FieldFit lens	✓ live warp	Lens lookup tables

Glossary

Chart: a room or cell with its own local coordinates.

Gluing: the rule joining two charts (portal, glued wall, branch cut, time seam).

Exp / log map: take a straight step in curved space / the direction and distance to a point.

Holonomy: how far a carried arrow turns after a loop (curvature × area).

Klein chart: a way to draw hyperbolic space where straight lines stay straight.

HEALPix: an equal-area grid on the sphere (49,152 directions here).

ITV: the space a tumour sweeps through over the breathing cycle.

Late warp: rotating the finished image by the newest head motion just before display.

Memoryless MSAA: anti-aliasing kept in on-chip tile memory, never written out.

Stencil portal: a doorway drawn by masking pixels and re-drawing the far side inside them.

Render Graph: URP's way of declaring passes so Unity schedules them and their memory.

asmdef: a Unity assembly definition; ours enforce which modules may reference which.

IL2CPP: Unity's C# → C++ build path, required on iOS.

Motion-to-photon: time from a head movement to the matching light on screen.

> **Hand to the implementation agent:** `ATLAS_Engineering_Spec.md` (this plan in full detail: interfaces, pseudo-code, schemas, pipelines, spikes) together with the design brief `ATLAS_Implementation_Brief.md`, the research dossier and the `illustrations/` folder. Version numbers marked [verify] in the spec must be checked against the pinned Unity release.